ASEAN
GEOAI FUSION 2026

Python Geospatial Primer

PART I: PYTHON BASICS, GEOPANDAS & VECTOR DATA

Spatial Operations & Vector Data Focus

ASEAN
GEOAI FUSION 2026

LEARNING OBJECTIVES

Understand why Python is used for geospatial analysis and identify key libraries in the ecosystem.

Describe the role of vector data and explain how GeoPandas extends Pandas for spatial workflows.

Apply spatial operations including buffers, joins, and overlays to solve real-world GIS problems.

01 Foundations

02 GeoPandas

03 Spatial Operations

Ecosystem Role

Understand Python's role and advantages in modern geospatial analytics.

GeoPandas Architecture

Read, structure, and manipulate spatial datasets systematically.

Vector Processing

Work directly with standard vector data formats and coordinates.

Spatial Operations

Execute fundamental geographic queries and transformations.

ASEAN
GEOAI FUSION 2026

WHY PYTHON FOR GEOSPATIAL?

Open-source, powerful, and extensible — Python powers the modern geospatial workflow from data loading to spatial analysis and mapping.

GeoPandas

Extends Pandas with geometry support. Read, write, and analyze vector data (points, lines, polygons) with ease.

Use: Spatial joins, overlays, CRS management, attribute queries.

Rasterio

Reads and writes raster datasets (GeoTIFF, NetCDF). Provides band-level access and affine transforms.

Use: Satellite imagery, DEMs, raster masking & clipping.

Folium

Creates interactive Leaflet.js web maps from Python. Visualize GeoDataFrames and GeoJSON layers in a browser.

Use: Interactive choropleth maps, marker clusters, tooltips.

Shapely & PyProj

Shapely: geometric operations (buffer, union, intersect). PyProj: coordinate reference system transformations and projections.

Use: Geometry construction, CRS reprojection, distance calc.

01 Python Foundations

Mapping standard data types to spatial attributes.

Foundational Data Structures

Variables & Attributes

```
city = "Kuala Lumpur"
pop = 1800000
```

Strings and Integers form the core of spatial attribute tables.

Coordinate Lists

```
states = ["Selangor",
"Johor"]
```

Lists group spatial boundaries and sequence coordinate pairs.

Dictionaries as Metadata

```
city = {"name": "KL",
"pop": 1800000}
```

Dictionaries natively structure complex metadata and spatial properties.

ASEAN
GEOAI FUSION 2026

LISTS AND DICTIONARIES

Python's core data structures are essential for storing and managing geospatial data. Lists and dictionaries allow us to organise coordinates, metadata, and attribute information efficiently. Understand how each structure is used in geospatial workflows:

Lists

An ordered, mutable sequence used to store multiple values such as coordinates or a series of feature attributes.

E.g. coords = [(-33.87, 151.21), (-37.81, 144.96)]
zone_ids = [101, 102, 103, 104]
route = [[x1,y1], [x2,y2], [x3,y3]]

Dictionaries

An unordered collection of key-value pairs used to store named attributes, metadata, or feature properties.

E.g. feature = {"name": "Sydney", "lat": -33.87, "lon": 151.21, "pop": 5300000}

02

GeoPandas Architecture

Extending standard dataframes for spatial analysis.

ASEAN
GEOAI FUSION 2026

FUNCTIONS AND PACKAGES

Functions allow you to write reusable, modular code that can be called multiple times throughout your workflow. Python packages extend functionality by providing pre-built tools for geospatial analysis, data handling, and visualization.

Defining Functions

Functions encapsulate reusable logic using the def keyword. They accept parameters and return results, making your geospatial workflows cleaner and easier to maintain.

```
def reproject_layer(gdf, crs):
    return gdf.to_crs(crs)
def buffer_features(gdf, dist):
gdf['geometry'] = gdf.buffer(dist)
    return gdf
def load_and_filter(path, col, val):
    data = gpd.read_file(path)
    return data[data[col] == val]
```

Importing Packages

Python packages are imported using import statements. Key geospatial packages include GeoPandas, Shapely, Fiona, PyProj, and Folium for mapping and spatial analysis.

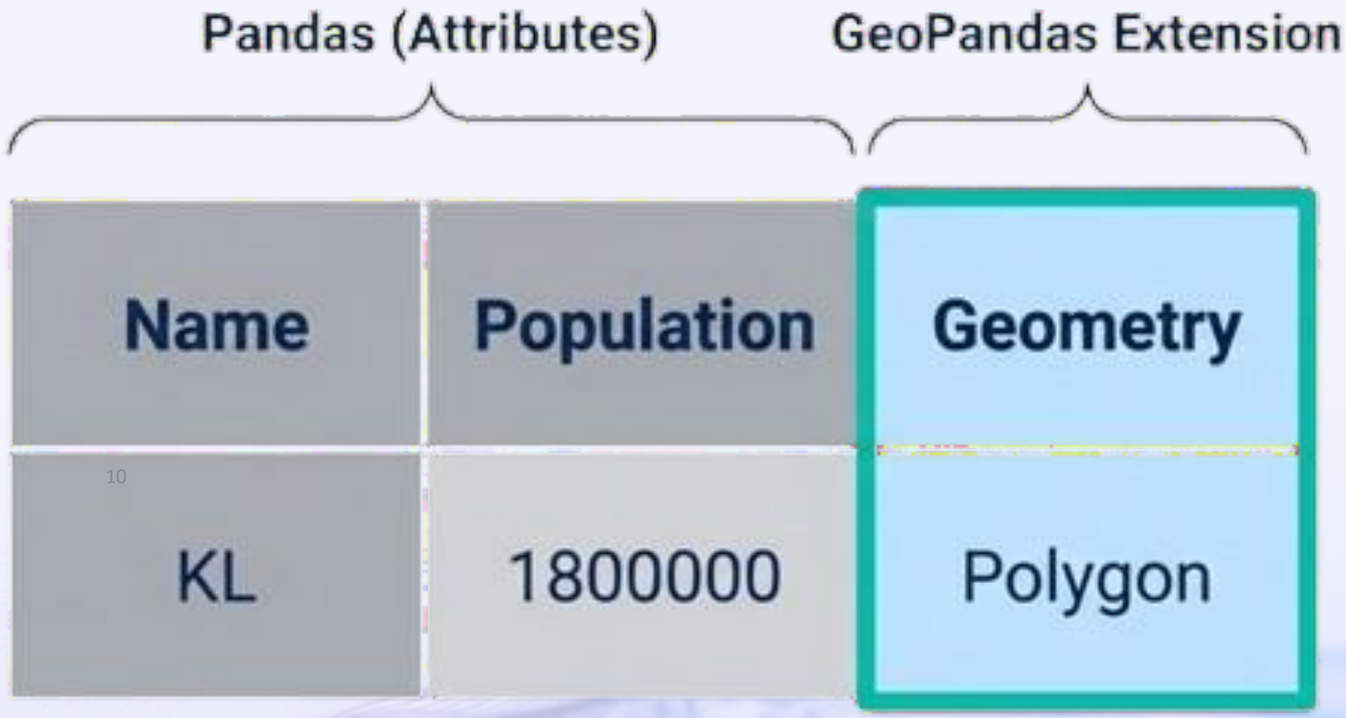
```
import geopandas as gpd
import pandas as pd
import matplotlib.pyplot as plt
from shapely.geometry import Point
from pyproj import CRS
Automate full workflow:gdf = load_and_filter('data.shp', 'type',
'school')
gdf_proj = reproject_layer(gdf, 'EPSG:3857')
```

THE GEOMETRY ANCHOR

GeoSeries: Stores geometries exclusively.

GeoDataFrame: Combines Attributes + Geometries + Coordinate Reference Systems (CRS).

"Think of a GeoDataFrame as an Excel spreadsheet with a dedicated geometry column."



ASEAN
GEOAI FUSION 2026

GEOSERIES & GEODATAFRAME

GeoPandas extends Pandas with two core spatial data structures. Understanding these is fundamental to all geospatial workflows in Python.

GeoSeries

A Pandas Series that holds geometry objects (Points, Lines, Polygons). Each element is a Shapely geometry. Supports spatial methods directly on the series.

E.g. gdf['geometry'] returns a GeoSeries of all shapes.

GeoDataFrame

A Pandas DataFrame with a special geometry column. Inherits all DataFrame operations plus spatial methods like .area, .distance(), and .plot().

E.g. gdf = gpd.GeoDataFrame(df, geometry='geom')

Geometry Column

The designated column storing Shapely geometry objects. Set via set_geometry(). A GeoDataFrame can hold multiple geometry columns but only one is active at a time.

E.g. gdf.set_geometry('new_geom') switches active geometry.

CRS

Coordinate Reference System defines how geometries map to Earth's surface. Stored as gdf.crs. Use .to_crs() to reproject between systems (e.g. WGS84 to UTM).

E.g. gdf.to_crs(epsg=32754) reprojects to GDA2020 / MGA zone 54.

ASEAN
GEOAI FUSION 2026

GEOSERIES & GEODATAFRAME

GeoPandas introduces two core data structures that extend Pandas to handle spatial data. Understanding these structures is essential for working with vector data in Python.

GeoSeries

A GeoSeries is a one-dimensional array of geometry objects (Points, Lines, Polygons). It extends Pandas Series and supports spatial operations like area, length, centroid, and distance calculations.

E.g. gdf.geometry → returns the active GeoSeries
gdf.crs → returns the Coordinate Reference System (e.g. EPSG:4326)

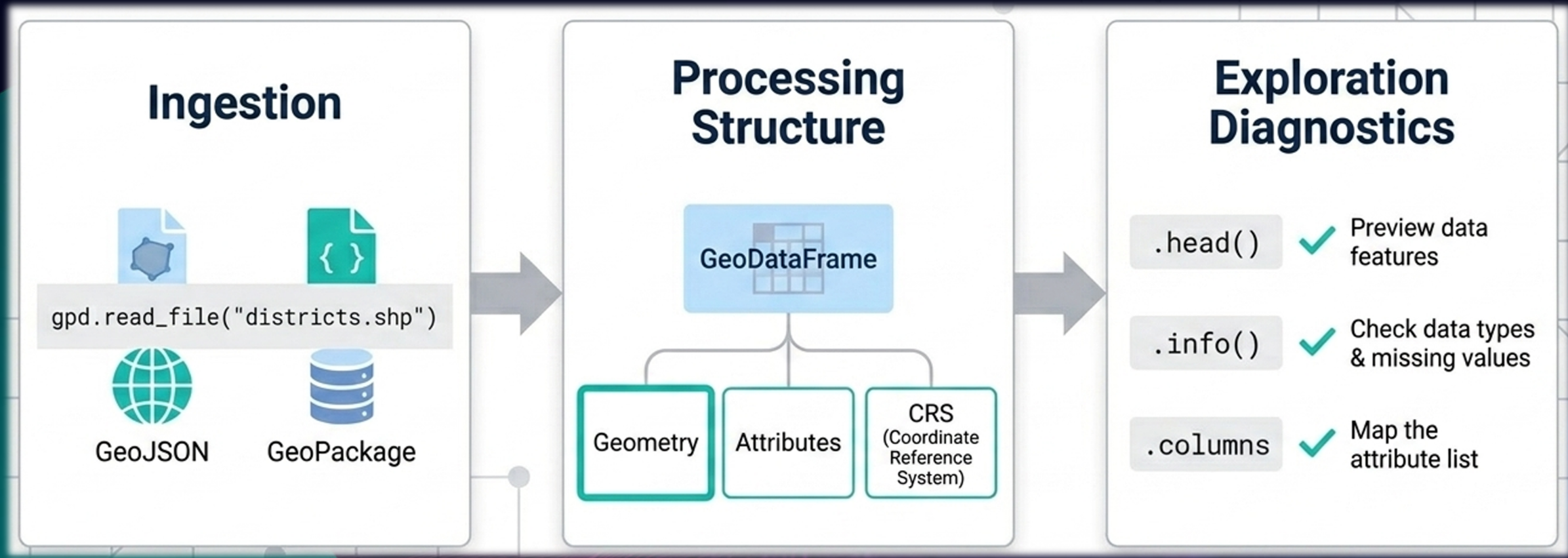
GeoDataFrame

A GeoDataFrame is a tabular data structure with one special geometry column (a GeoSeries). Each row represents a spatial feature with attributes — similar to a Shapefile attribute table.

E.g. gdf = gpd.GeoDataFrame(df, geometry='geom')
Sets the geometry column and enables spatial methods on the full table.

ASEAN
GEOAI FUSION 2026

DATA INGEETION AND EXPLORATION PIPELINE



ASEAN
GEOAI FUSION 2026

READING VECTOR DATA

GeoPandas uses geopandas.read_file() to load vector data from multiple formats. Understanding each format helps you choose the right one for your geospatial workflow.

Shapefile

A legacy ESRI format using multiple files (.shp, .dbf, .prj). Widely supported across GIS tools.

```
gdf = gpd.read_file("roads.shp")
print(gdf.head())
print(gdf.crs)
```

GeoJSON

A lightweight, text-based format using JSON. Ideal for web mapping and sharing spatial data.

```
gdf = gpd.read_file("data.geojson")
print(gdf.geometry.type)
```

GeoPackage

A modern OGC standard using a single SQLite file. Supports multiple layers and large datasets.

```
gdf = gpd.read_file("data.gpkg",
layer="boundaries")
print(gdf.info())
```

ASEAN
GEOAI FUSION 2026

VISUALIZING VECTOR DATA

Plot GeoDataFrames using .plot() to render points, lines, and polygons directly from spatial data.

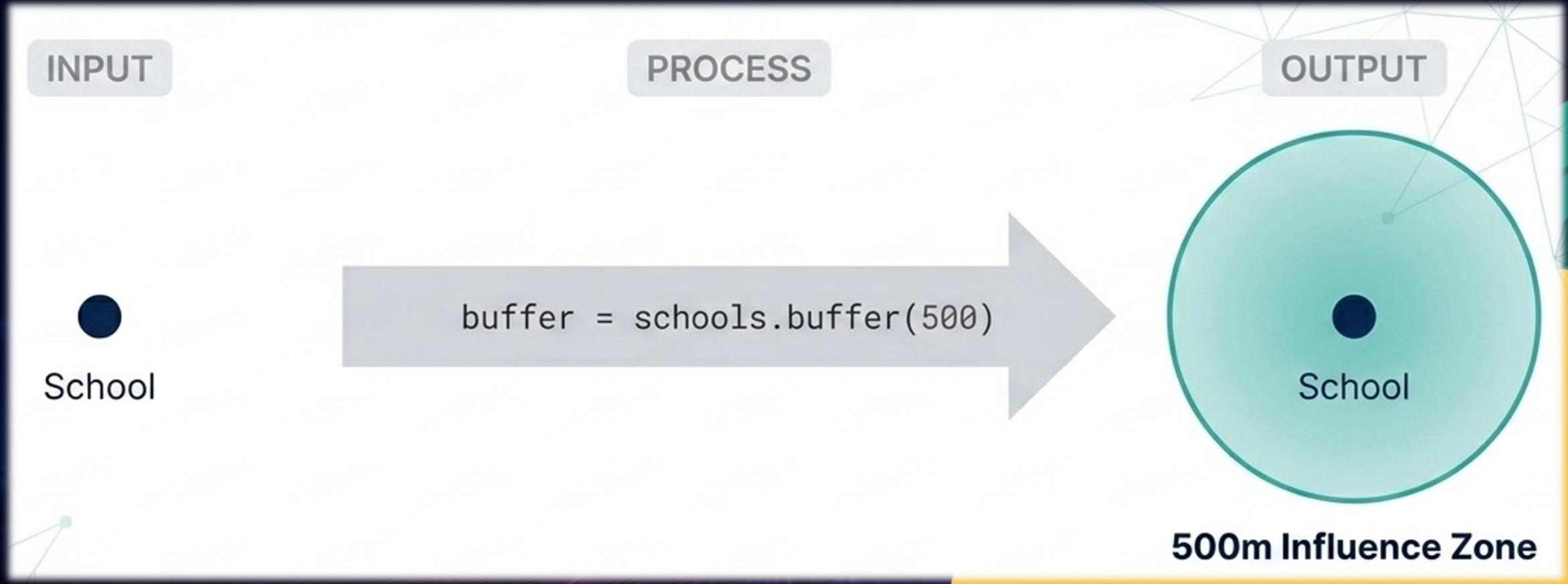
Create choropleth maps by mapping attribute values to color gradients for thematic analysis.

Apply color schemes, legends, and titles for clear, publication-ready thematic visualization.

03 Spatial Operations

Deploying analytical tools to solve geographic problems.

BUFFER ANALYSIS



ASEAN
GEOAI FUSION 2026

SPATIAL JOIN

A spatial join merges attributes from two datasets based on their geographic relationship. It answers:
"Which features from one layer share a location with features from another?"
Two common spatial join types used in GIS analysis:

Join by Location

Combines attributes of two layers where geometries intersect, contain, or are within each other.

E.g. Joining school points to district polygons — each school inherits the district name, ID, and region code from the polygon it falls within.

Nearest Join

Matches each feature to the closest feature in another layer, even if they do not directly overlap or intersect.

E.g. Assigning each school to its nearest health clinic or bus stop — useful for accessibility and service coverage analysis.

ASEAN
GEOAI FUSION 2026

OVERLAY & INTERSECTION

Overlay operations combine two or more vector layers to analyse spatial relationships. Key operations include intersection, union, and difference — essential for tasks like flood-risk mapping and land-use analysis.

Overlay Operations

Overlay functions in GeoPandas allow you to combine polygon layers spatially:

- Intersection – keeps only overlapping areas
- Union – merges all features from both layers
- Difference – removes overlapping areas from one layer

E.g. geopandas.overlay(flood_zones, parcels, how='intersection') — returns only the areas where flood zones and parcels overlap.

Flood-Risk Use Case

Intersect a flood-extent polygon layer with a buildings or parcels layer to identify at-risk properties:

- Input: flood zones + land parcels
- Output: parcels within flood extent
- Use: emergency planning, risk assessment

E.g. geopandas.overlay(city_boundary, river_buffer, how='difference') — subtracts the river buffer from the city boundary to show buildable land outside flood risk zones.

ASEAN
GEOAI FUSION 2026

KEY TAKEAWAYS

Recap: Python basics, GeoPandas, reading & exploring vector data, spatial joins, buffers, and overlay operations.

Competencies Gained: Load, inspect, visualize, and analyze vector geospatial data using Python and GeoPandas workflows.

Next Session: Raster data, coordinate systems, projections, and advanced spatial analysis with Python.